
Current formula — what it is and what's wrong with it

```
likelihood = anomaly_score * 10.0          # [0, 10]
impact     = (intent_score * 0.5) + (log10(event_count)) # [0, 10]
severity   = likelihood * (impact / 10.0)      # [0, 10]
```

This is a **multiplicative product formula** — **impact/10** acts as a scaling factor on **likelihood**. It works acceptably for BETH but breaks entirely for DARPA APT. Here is the exact failure mode:

Scenario	anomaly_score	event_count	intent	likelihood	impact	severity
BETH botnet (correct HIGH)	0.85	4000	7	8.5	7.1	6.0 ✓
APT sshd injection (should be CRITICAL)	0.90	8	5	9.0	3.4	3.1 ✗
APT C2 + kernel module (should be CRITICAL)	0.88	12	7	8.8	4.6	4.1

The APT sshd injection gets severity 3.1 — **below your own threshold of 4.0** — even with a 90% anomaly score. You would miss it entirely. This is not a minor calibration issue; it is a structural flaw for APT detection.

Root cause: **blast_radius = log10(event_count)** was designed for BETH where volume correlates with impact. APT is the inverse — stealth IS the attack. A kernel rootkit that touches 8 processes is more dangerous than a botnet touching 4000. Your formula penalises the APT for being stealthy.

Is the current calculation the best that exists? No.

The existing formula is an informal OWASP-style risk rating. The academic standard for provenance-graph security scoring has evolved significantly:

What papers actually use:

- **CVSS v3.1** — multiplicative but separates base/temporal/environmental, scope change modifier. Still widely cited but criticised for non-linearity failures.
 - **Attack Severity Index (ASI)** from APT provenance papers — additive weighted sum of kill-chain-stage signals. Used in KAIROS, ThreaTrace evaluation suites.
 - **Kill Chain Stage Scoring** — each MITRE tactic maps to a kill chain stage (1-7), and severity increases monotonically with stage. Persistence > Initial Access even at the same event count.
 - **CVSSv4.0 (2023)** — most recent, adds Threat/Environmental modifiers, moves away from the pure multiplicative structure.
-

The right answer: two separate impact calculators, one shared formula structure

You should **not** use the same impact calculation for both datasets. But you **should** use the same final formula structure — additive, not multiplicative.

Why additive and not multiplicative:

In the current formula, $\text{impact}/10$ suppresses likelihood when impact is low. A multiplicative formula requires BOTH components to be high to produce a high score. For APT, you want a high ML score to produce a high severity score regardless of footprint. An additive blend respects both signals independently.

Recommended formula

import math

```
KILL_CHAIN_STAGE = {
    "T1189": 1, # Initial Access — Drive-by
    "T1059": 2, # Execution
    "T1068": 3, # Privilege Escalation
    "T1055": 4, # Defence Evasion / Process Injection
    "T1547": 5, # Persistence
    "T1014": 5, # Persistence — Rootkit
    "T1071": 6, # Command & Control
    "T1041": 7, # Exfiltration
    "T1003": 4, # Credential Access
    "T1070": 3, # Defence Evasion
    "T1078": 2, # Initial Access — Valid Accounts
}
```

```

def compute_severity(metrics, dataset_tag):
    """
    Unified additive severity formula with dataset-specific impact components.
    Returns (final_severity [0-10], likelihood [0-10], impact [0-10], tier str)
    """
    # — Component 1: Likelihood
    

---


    # Direct from ML ensemble — already the best trained signal you have.
    # Scale to [0, 10]. No transformation needed.
    likelihood = round(metrics["anomaly_score"] * 10.0, 3)

    # — Component 2: Impact — dataset-specific
    

---


    if dataset_tag == "beth":
        impact = _beth_impact(metrics)
    else:
        impact = _darpa_impact(metrics)

    # — Final severity: weighted additive blend 

---


    #  $\alpha=0.60$  weights likelihood (trained ML) slightly higher.
    #  $\beta=0.40$  weights impact (context signals).
    # Additive: high likelihood ALWAYS contributes, even if impact is moderate.
    # This prevents the multiplicative suppression that hides APT in stealthy cases.
    ALPHA = 0.60 # likelihood weight
    BETA = 0.40 # impact weight

    raw = (ALPHA * likelihood) + (BETA * impact)
    final_severity = round(min(raw, 10.0), 1)

    # — 3-tier verdict
    

---


    if final_severity < 3.5:
        tier = "BENIGN"
    elif final_severity < 6.5:
        tier = "SUSPICIOUS"
    else:
        tier = "MALICIOUS"

    return final_severity, likelihood, impact, tier

def _beth_impact(metrics):
    """

```

BETH: volume-aware impact.

Botnet/cryptomining attacks leave large event footprints.

blast_radius is meaningful here.

"""

Base TTP impact from your existing MITRE map (1-5 scale assumed)

intent = float(metrics.get("ttp_impact", 1))

Static signals

if metrics.get("rules_matched"): intent += 2.0

if metrics.get("sensitive_paths"): intent += 3.0

if metrics.get("is_root"): intent += 2.0

Blast radius — valid for BETH because attack volume IS correlated with danger

$\log_{10}(1)=0$, $\log_{10}(10)=1$, $\log_{10}(1000)=3$, $\log_{10}(100000)=5$

blast = math.log10(max(metrics.get("event_count", 1), 1))

impact = (intent * 0.55) + (blast * 0.9)

return round(min(impact, 10.0), 3)

def _darpa_impact(metrics):

"""

DARPA THEIA APT: kill-chain-stage-aware, stealth-aware impact.

Blast radius is REMOVED — small footprint does not reduce severity.

C2 connection and injection events are the primary amplifiers.

"""

ttp_id = metrics.get("ttp_id", "")

Kill chain stage: position in the attack progression [1-7] → scaled to [1.4-10]

stage = KILL_CHAIN_STAGE.get(ttp_id, 1)

kill_chain_score = stage * (10.0 / 7.0) # normalise to [1.43, 10.0]

C2 confirmed connection — highest single impact signal for APT

Connecting to a known C2 is unambiguously malicious regardless of volume

c2_bonus = 4.0 if metrics.get("has_c2_connection") else 0.0

Injection events (ptrace on sshd etc.)

inject_bonus = 2.5 if metrics.get("inject_count", 0) > 0 else 0.0

Direct C2 event count (outbound connections to malicious IPs)

c2_event_bonus = min(metrics.get("c2_event_count", 0) * 0.5, 2.0)

Root privilege

root_bonus = 1.5 if metrics.get("is_root") else 0.0

```

# Rules matched (APT signatures)
rule_bonus = 1.5 if metrics.get("rules_matched") else 0.0

impact = kill_chain_score + c2_bonus + inject_bonus + c2_event_bonus + root_bonus +
rule_bonus
return round(min(impact, 10.0), 3)

```

What you need to add to **metrics** collection in the TAO loop

Your current **metrics** dict collects BETH signals fine. For DARPA you need to add extraction in the tool result parsing section:

```

# Inside the SequenceScorer result parsing block:
if tool_name == "SequenceScorer":
    details = parsed_result.get("details", {})
    metrics["anomaly_score"] = max(...) # existing
    metrics["event_count"] = details.get("event_count", 0)
    metrics["dataset_tag"] = details.get("dataset", "beth") # NEW
    metrics["inject_count"] = details.get("inject_count", 0) # NEW
    metrics["c2_event_count"] = details.get("c2_event_count", 0) # NEW

# Inside the ContextRetriever result parsing block:
elif tool_name == "ContextRetriever":
    files = res_data.get("FilesAccessed", [])
    if any("shadow" in f or "/etc/" in f for f in files):
        metrics["sensitive_paths"] = True
    metrics["has_c2_connection"] = res_data.get("has_c2_connection", False) # NEW

# Inside the TTPMapper result parsing block:
elif tool_name == "TTPMapper":
    for key, val in MITRE_TTP_SEVERITY_MAP.items():
        if key in str(res_data):
            metrics["ttp_impact"] = val["impact"]
            metrics["ttp_id"] = key # NEW — needed for kill chain stage lookup
            break

```

Then replace the entire Phase 3 block with:

```
# Phase 3: severity calculation
dataset_tag = metrics.get("dataset_tag", "beth")
final_severity, likelihood, impact, tier = compute_severity(metrics, dataset_tag)
```

Threshold change: 2-level → 3-level

Your current benchmark grades `severity >= 4.0` as malicious (binary). With the new additive formula and APT support you need three tiers:

```
# In run_benchmark grading logic:
if final_severity >= 6.5:
    pred_label = 2 # MALICIOUS
elif final_severity >= 3.5:
    pred_label = 1 # SUSPICIOUS
else:
    pred_label = 0 # BENIGN
```

```
# Map DARPA ground truth to same scale:
# evil > 0 → label=2, sus > 0 and evil=0 → label=1, else → label=0
```

This matters because for DARPA, a SUSPICIOUS verdict on a process that's part of an APT kill chain is more informative than a binary benign/malicious split — and the TAO loop can continue investigating suspicious cases.

Summary of all changes

	Current	Recommended
Formula type	Multiplicative $L \times (I/10)$	Additive $0.6L + 0.4I$
Impact for BETH	intent + blast_radius	Same, coefficients tuned
Impact for DARPA	Same as BETH (wrong)	Kill chain stage + C2 + inject signals; no blast radius
APT stealth	Penalised (small footprint = low score)	Rewarded (C2 + stage dominate)

Thresholds	Binary: ≥ 4.0 = malicious	3-tier: <3.5 benign, 3.5–6.5 suspicious, ≥ 6.5 malicious
New metrics needed	None	<code>ttp_id</code> , <code>has_c2_connection</code> , <code>inject_count</code> , <code>dataset_tag</code>